

Introduction to Deep Learning - A0

Deepak Somesh K J (s4587243)

Nallathambi Vethiappan (s4577663)

Divyanshi Singh (s4374827)

1 Introduction

Pattern recognition and classification have been central to machine learning since the mid-20th century. The perceptron, was one of the first algorithms to learn category boundaries from features. At the same time, distance-based methods such as nearest-neighbor provided simple yet effective baselines, especially in high-dimensional spaces.

This assignment covers three tasks:

- **Task 1:** Distance-based classifiers (class centers, NMC, KNN) and dimensionality reduction for visualization.
- **Task 2:** Multi-class perceptron trained with gradient descent, illustrating linear separability limits.
- **Task 3:** A simple feedforward neural network trained to solve the XOR problem.

Each task includes results, visualizations, and analysis, followed by a comparison of Task 1 and Task 2. The overall aim is to implement these models, evaluate them on a given dataset, and interpret their performance.

2 Dataset Description

The dataset used in this assignment consists of grayscale images of handwritten digits, each flattened into a 256-dimensional feature vector. The outputs are integer labels from 0 to 9, corresponding to the digit classes.

Sizes and Shapes

- **Train set:** 1,707 samples $\times$ 256 features
- **Test set:** 1,000 samples $\times$ 256 features
- **Number of classes:** 10 (digits 0–9), with their distribution across training and test sets detailed in Table 1.
- **Feature range:** $[-1.0, 1.0]$ for both train and test
- **Data types:** X as float64, y as int64
- **Missing values:** None in features or labels

For the comparisons, **all experiments were done using raw input features**. This ensures that Task 1 and Task 2 operate on the same input representation, making their accuracies directly comparable.

Table 1: Class counts in training and test sets

Class	Train Count	Test Count
0	319	224
1	252	121
2	202	101
3	131	79
4	122	86
5	88	55
6	151	90
7	166	64
8	144	92
9	132	88

3 Task 1 – Data dimensionality, Distance-based Classifiers

3.1 Class Centers and Inter-class Distances

Each class center was computed as the mean vector of all training samples in that class. The resulting 10 centers were compared using Euclidean distance.

3.1.1 Results

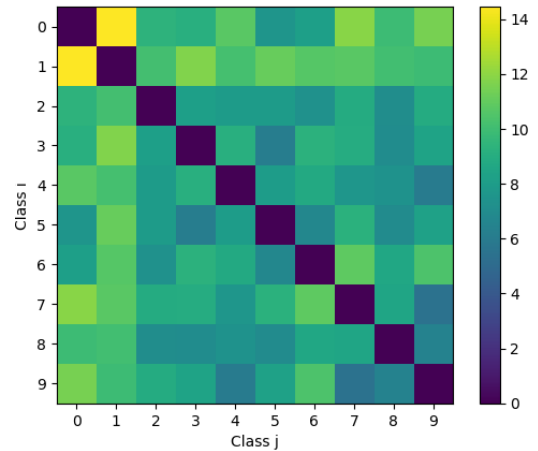


Figure 1: Inter-class Euclidean distance heatmap between class centers

The inter-class Euclidean distance heatmap in Figure 1 shows the distances between the 10 centers, with clos-

est pairs (7,9)=5.43, (4,9)=6.01, (3,5)=6.12, (8,9)=6.40, (5,6)=6.70, aligning with human handwriting confusions like 9 resembling 4 or 7, and 3 looking like 5. Most distinct pairs, such as 0 vs 1 and 1 vs 8, reflect very different shapes.

3.1.2 Analysis

If the average appearances of two digits are close, then distance-based classifiers will struggle. Later confusion matrices confirm this expectation.

3.2 Dimensionality Reduction

To gain insight into the data geometry, three dimensionality reduction techniques were applied to project the 256-dimensional inputs into 2D space. Dimensionality reduction allows visualization of high-dimensional structure. We used PCA, t-SNE, and UMAP.

3.2.1 Results

- **PCA (2D):** The first two principal components, depicted in the PCA 2D projection of Figure 2, explain around 28.5% of total variance, revealing visible clusters that overlap heavily, especially among digits 4, 9 and 3, 5.
- **t-SNE (2D):** A nonlinear method preserving local neighborhoods [5], the resulting 2D scatter plot depicted in the t-SNE 2D projection of Figure 3 showed tighter digit clusters, with improved separation compared to PCA, though some classes still overlapped.
- **UMAP (2D):** The UMAP plot [2] plot depicted in the UMAP 2D projection of Figure 4 produces compact clusters, visually separating digits more clearly than PCA.

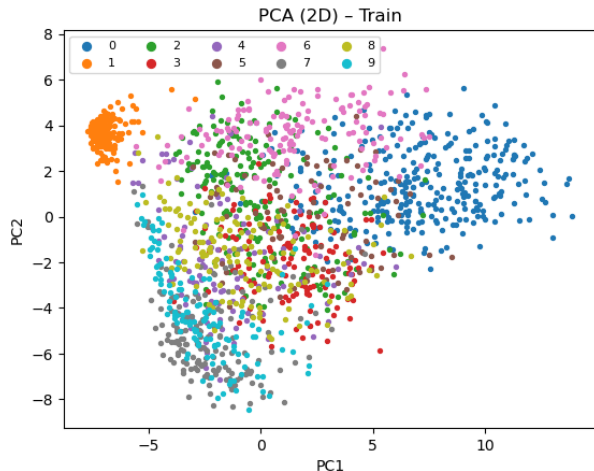


Figure 2: PCA 2D projection

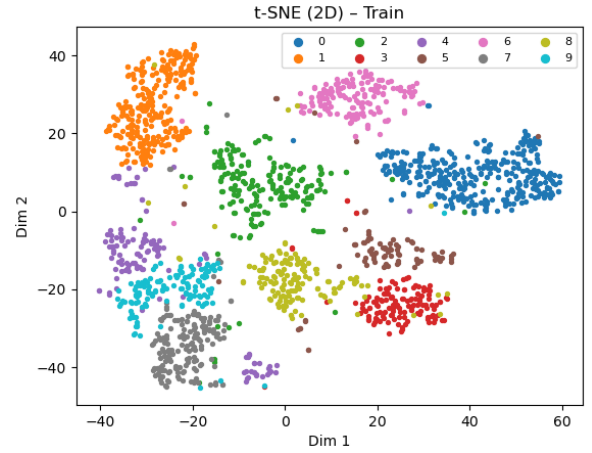


Figure 3: t-SNE 2D projection

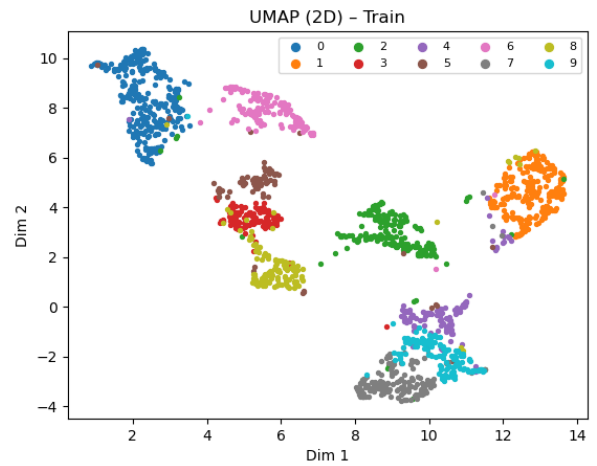


Figure 4: UMAP 2D projection

3.2.2 Analysis

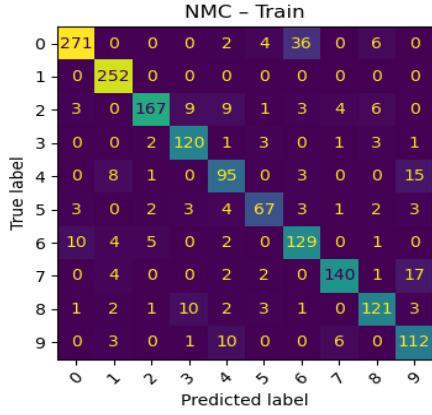
These plots illustrate how different methods uncover structure in the data, and why dimensionality reduction is often used for visualization and preprocessing.

3.3 Nearest Mean Classifier (NMC)

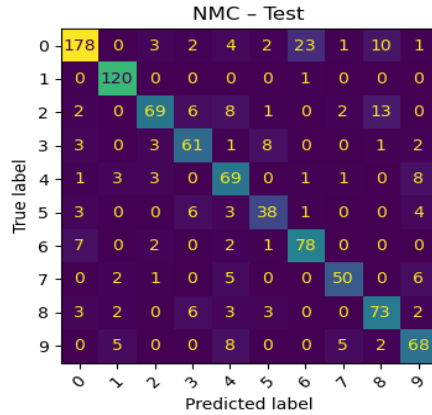
The Nearest Mean Classifier assigns a new input to the class whose mean vector is closest in Euclidean distance. It is computationally efficient, requiring only one mean vector per class.

3.3.1 Results

Train accuracy: **0.8635**. Test accuracy: **0.8040**. The confusion patterns, illustrated in the NMC confusion matrix of Figure 5, highlight concentrations on digit pairs 4↔9, 3↔5, and 7↔9, with the matrix revealing darker off-diagonal elements that indicate frequent misclassifications between these similar shapes.



(a) NMC confusion matrix – train



(b) NMC confusion matrix – test

Figure 5: NMC confusion matrices

Analysis. NMC’s reliance on single prototypes per class makes it a weak but interpretable baseline. Its errors directly reflect inter-class closeness from Step 1.

3.4 K-Nearest Neighbors (KNN)

The KNN classifier predicts a class by majority voting among the k nearest training samples, as formalized by Cover and Hart [1]. Distance weighting was applied, giving closer neighbors higher influence.

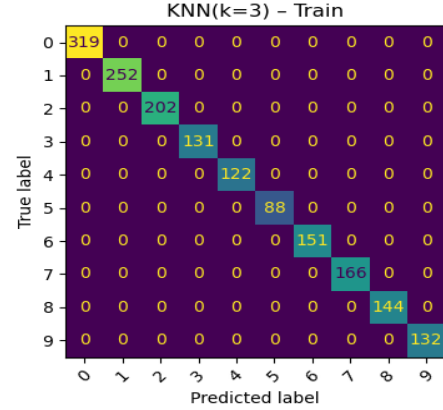
Experiments with $k \in \{1, 3, 5, 7\}$ were conducted.

3.4.1 Results

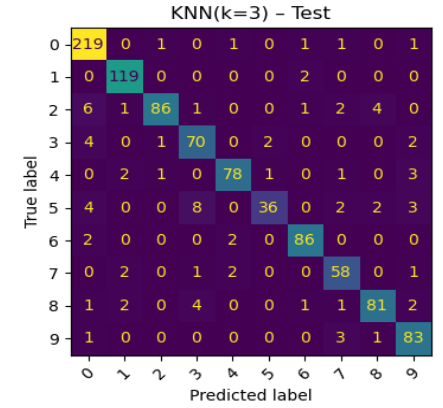
- $k = 1$: train 1.0000, test 0.9150
- $k = 3$: train 1.0000, test 0.9160 (best)
- $k = 5$: train 1.0000, test 0.9160
- $k = 7$: train 1.0000, test 0.9160

The confusion matrix, presented in the KNN confusion matrix of Figure 6, is nearly diagonal, with residual errors confined to visually similar pairs such as $4 \leftrightarrow 9$ and $3 \leftrightarrow 5$,

highlighting the classifier’s challenge with shape ambiguities.



(a) KNN confusion matrix – train.



(b) KNN confusion matrix – test.

Figure 6: KNN confusion matrices for $k = 3$

3.4.2 Analysis

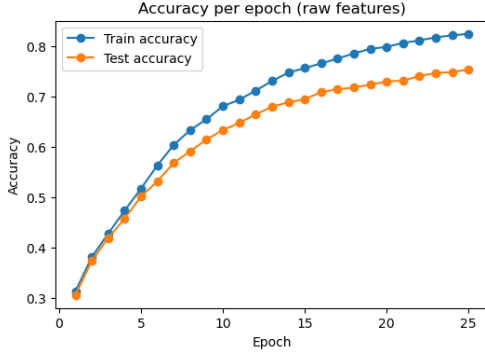
KNN achieves the best performance (91.6% test accuracy). Memorization explains perfect training accuracy; generalization is strong because local neighborhoods are truly digit-specific. The main drawback is computational cost at inference.

4 Task 2 – Multi-class Perceptron

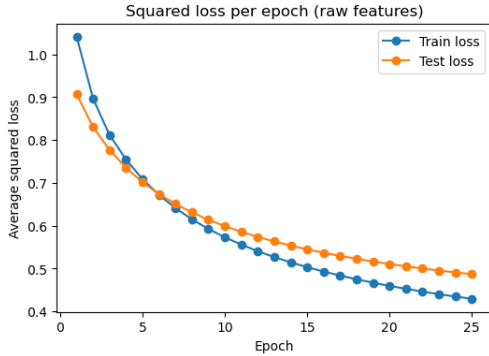
The perceptron is a linear model that computes weighted sums of inputs and applies a decision rule, as introduced by Rosenblatt [4]. In the multi-class case, weights are organized into a matrix, one column per class. The perceptron predicts the class with the maximum score. In this assignment, it was trained using squared-loss gradient descent, with weight updates guided by the rules detailed in Chapter 5 of Understanding Deep Learning [3], ensuring smooth optimization.

4.1 Training dynamics

Train accuracy, depicted in the perceptron accuracy curves of Figure 7(a), rose from ~ 0.38 (epoch 1) to ~ 0.82 (epoch 25); test accuracy followed a similar trend in Figure 7(a), increasing from ~ 0.31 to ~ 0.75 . Loss, shown in the perceptron squared loss curves of Figure 7(b), decreased smoothly (train: $1.04 \rightarrow 0.43$; test: $0.97 \rightarrow 0.49$), indicating stable convergence with a modest generalization gap (~ 7 pp).



(a) Perceptron accuracy per epoch (raw features)



(b) Perceptron squared loss per epoch (raw features)

Figure 7: Perceptron training curves

4.2 Final results

- Final train accuracy: 0.824
- Final test accuracy: 0.753
- Best test accuracy: 0.753

Confusion analysis. In figure 8, Digits 0 and 1 are classified almost perfectly. Errors concentrate on $5 \leftrightarrow 3/8$, $9 \leftrightarrow 4/7$, $6 \leftrightarrow 5$, matching closeness pairs from Task 1.

4.3 Hyperparameter sensitivity

In figure 9, we tested four learning rates across five random seeds.

- $lr=0.001 \rightarrow$ mean test acc 0.462

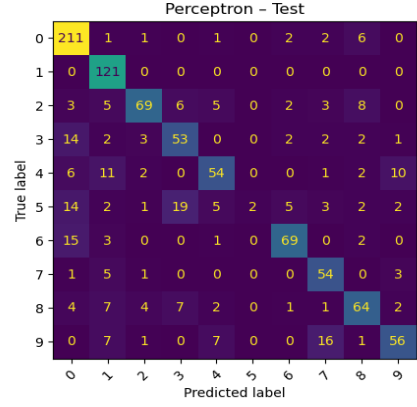


Figure 8: Perceptron confusion matrix - test

- $lr=0.002 \rightarrow$ mean 0.609
- $lr=0.005 \rightarrow$ mean 0.765
- $lr=0.01 \rightarrow$ mean 0.820 (best)

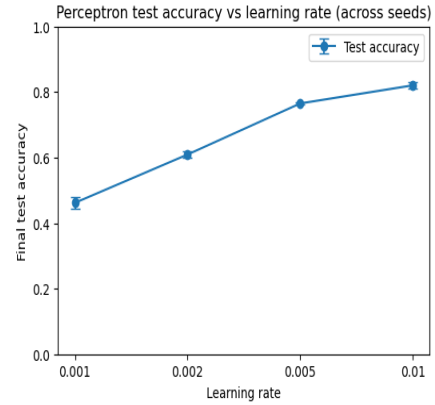


Figure 9: Perceptron test accuracy vs. learning rate with error bars

Performance improves sharply with larger learning rates, plateauing near 0.82 accuracy. The perceptron is sensitive to step size but robust across seeds once lr is chosen well.

5 Comparative Analysis: Task 1 vs Task 2

The results shown in Table 2 highlight the difference between the distance-based models (NMC and KNN) and the perceptron how these approaches exploit the structure of the data. The outputs reveal trade-offs between accuracy, generalization, and computational efficiency.

Summary

- Best accuracy: KNN (91.6%).

Table 2: Accuracy comparison (Task-1 vs Task-2)

Model	Description	Test Acc.
Task 1 (k-NN, k=3)	Final test accuracy	0.9160
Task 2 (Perceptron)	Final test accuracy	0.7530
Task 2 (Perceptron)	Best test accuracy	0.7530

- **Most efficient at inference:** Perceptron (75.3%).
- **Baseline:** NMC (80.4%).

This comparison highlights a fundamental trade-off. Distance-based classifiers exploit the strong local cluster structure of the dataset, achieving higher accuracy. Linear classifiers like the perceptron are computationally lighter but cannot capture the non-linear decision boundaries required for digit classification.

6 Task 3 – XOR network and the Gradient Descent

The XOR problem cannot be solved by a single-layer perceptron because it is not linearly separable. To address this, we implemented a feedforward network with two hidden neurons (sigmoid activations) and one output neuron. Training was performed using backpropagation with mean squared error. Inputs are the four binary pairs $[0, 0]$, $[0, 1]$, $[1, 0]$, $[1, 1]$ with XOR labels.

6.1 Results Across Runs

We trained the XOR network with five seeds and learning rates between 0.3 and 0.7. All runs converged to perfect classification (0/4 misclassified), though convergence speed varied. Table 3 summarizes the outcomes.

Table 3: Summary of XOR training across seeds and learning rates

Seed	Learning Rate	Epochs	Final MSE
0	0.5	6483	0.138
1	0.5	4185	0.122
2	0.5	3032	0.125
3	0.3	4952	0.131
4	0.7	1466	0.133

The table shows that while training time depends on initialization and learning rate, the final solution is consistent, the network always separates the XOR mapping.

6.2 Representative convergence behavior

A typical training run is shown in Figure 10. This example (seed=4, lr=0.7) demonstrates rapid convergence within 1500 epochs, with the mean squared error dropping from ~ 0.25 to ~ 0.13 and classification becoming perfect.

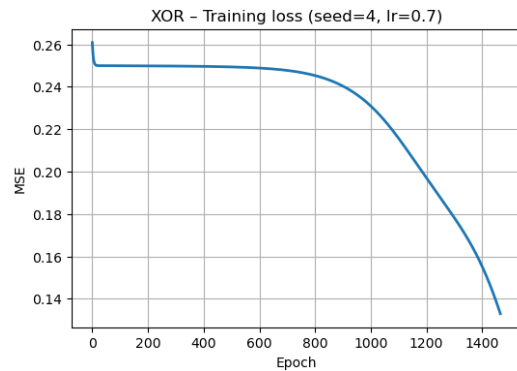


Figure 10: Training loss curve for XOR with seed=4, learning rate=0.7 (representative run)

6.3 Discussion

The XOR network learns a non-linear boundary beyond a perceptron’s capacity, with higher learning rates (lr=0.7 at ~ 1466 epochs vs. lr=0.5 at ~ 6483 epochs) speeding convergence. Variability across seeds reflects a non-convex loss surface, yet all runs achieved 0/4 misclassifications, affirming the 2–2–1 architecture’s robustness for XOR.

7 Comparative Analysis

Table 4: Summary of train/test accuracies

Model	Train acc	Test acc	Notes
NMC (Task 1)	0.8635	0.8040	Prototype baseline
KNN (k=3, Task 1)	1.0000	0.9160	Best performer
Perceptron (Task 2)	0.8240	0.7530	Linear boundary
XOR MLP (Task 3)	—	1.0000	Demonstrates non-linear separability

KNN excels because the dataset has strong local clusters. The perceptron underperforms due to non-linear boundaries. The XOR network confirms that hidden layers unlock non-linear separability.

8 Conclusion

This assignment demonstrated the evolution of classifiers, from the basic nearest mean classifier to the more robust KNN, which leverages local neighborhoods for a 91.6% digit classification accuracy, and the perceptron, a trainable linear model achieving ~ 75 – 82% and revealing linear limitations, to the MLP, which handles non-linear problems like XOR through hidden layers. These observations underscore both practical and theoretical insights into why certain algorithms outperform others, illustrating how modern neural networks build on these classical foundations.

References

- [1] Thomas M. Cover and Peter E. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27, 1967.
- [2] Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.
- [3] Simon J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023.
- [4] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.
- [5] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605, 2008.